EECS 182 Deep Neural Networks
Fall 2025 Anant Sahai and Gireeja Ranade

Final Exam

1. CNN Basics (10 pts)

We are going to describe a convolutional neural net using the following pieces:

- CONV(K, F, P, S) denotes a convolutional layer with F different filters, each of size $K \times K \times C$, where C is the depth (i.e. number of channels) of the activations from the previous layer. Padding is P, and stride is S.
- POOL(2) denotes a 2×2 max-pooling layer with stride 2 (pad 0)
- FLAT just turns whatever shape input tensor into a one-dimensional array with the same values in it.
- LIN(D) denotes a single fully-connected linear layer with D output activations. The number of input activations implicitly comes from the previous layer.

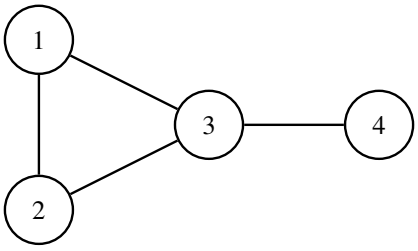
Note: All CONV and LIN layers have biases as well as weights. *Do not forget the biases when counting parameters.*

Now, we are going to use this network to do inference on a single input. **Fill in the missing entries in this table of the size of the number of parameters at each layer and the shape of the output activations at each layer. You can/should write your answer as a computation** (e.g. $128 \times 128 \times 3$) in the style of the already filled-in entries of the table. You don't have to evaluate it, but that can help check your work at the end.

Layer	Number of Parameters	Dimension of Activations in output of layer
Input	0	$26 \times 26 \times 1$
CONV$(5, 10, 1, 1)$	$5 \times 5 \times 1 \times 10 + 10$	$24 \times 24 \times 10$
POOL(2)	0	$12 \times 12 \times 10$
CONV$(3, 20, 1, 1)$	**Solution:** $3 \times 3 \times 10 \times 20 + 20$	**Solution:** $12 \times 12 \times 20$
CONV$(1, 20, 0, 2)$	**Solution:** $1 \times 1 \times 20 \times 20 + 20$	**Solution:** $6 \times 6 \times 20$
FLAT	0	720
LIN(10)	**Solution:** $720 \times 10 + 10$	10

2. DiffPool (12 pts)

Consider the following graph with 4 nodes:



 All Rights Reserved. This may not be publicly shared without explicit permission.

Suppose each node has 2-dimensional features given by feature matrix X where row i of X is the feature vector for node i:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 0 \end{bmatrix}$$

You will perform **one DiffPool layer**:

$$4 \text{ nodes } \rightarrow \text{ } 2 \text{ clusters.}$$

Suppose we additionally have GNNs which give the following (**unnormalized**) scores (in S) for each node and embeddings (in Z):

$$S = \begin{bmatrix} 2 & -\infty \\ 3 & -\infty \\ 1 & 1 \\ 0 & \ln 3 \end{bmatrix}, \qquad Z = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \\ 2 & 1 \end{bmatrix},$$

(a) (6pts) **Compute the node embeddings at the output of the diffpool:**

$$X^{(1)} = (S_{\text{softmax}})^\top Z.$$

Remember that $S_{softmax}$ takes the cluster scores for each node and turns them into a probability via the softmax operation (which takes the exponential of scores and then normalizes all those positive numbers into a probability).

Solution: First, we compute the row-wise softmax of S:

$$S_{\text{softmax}} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ \frac{1}{2} & \frac{1}{2} \\ \frac{1}{3+1} & \frac{3}{3+1} \end{bmatrix}.$$

Next, we compute the pooled node embeddings:

$$X^{(1)} = (S_{\text{softmax}})^\top Z.$$

Calculating this gives:

$$\begin{aligned} X &= \begin{bmatrix} 1 & 1 & \frac{1}{2} & \frac{1}{4} \\ 0 & 0 & \frac{1}{2} & \frac{3}{4} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 1 \\ 2 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1+0+\frac{1}{2}+\frac{2}{4} & 0+2+\frac{1}{2}+\frac{1}{4} \\ 0+0+\frac{1}{2}+\frac{6}{4} & 0+0+\frac{1}{2}+\frac{3}{4} \end{bmatrix} \\ &= \begin{bmatrix} 2 & 2.75 \\ 2 & 1.25 \end{bmatrix}. \end{aligned}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

(b) (6pts) Recall that to make training more effective, DiffPool adds auxiliary losses.

For each of the following auxiliary losses, **select the primary design purpose of using the auxiliary loss.**

i. LP loss: $\|A - S_{\text{softmax}} S_{\text{softmax}}^\top\|_F^2$ where A is the adjacency matrix of the graph.

◯ Align clusters to graph topology	◯ Encourage hard assignments of nodes to clusters	◯ Other

Solution: This encourages alignment of clusters to the graph topology. If the graph actually consisted of K fully-connected clusters with no edges between nodes in different clusters, then the natural assignment of nodes to clusters would result in zero loss under this metric.

ii. Average entropy minimization loss[1]: $-\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{K} S_{\text{softmax}}[i,j] \log S_{\text{softmax}}[i,j]$

◯ Align clusters to graph topology	◯ Encourage hard assignments of nodes to clusters	◯ Other

Solution: This encourages assignments close to hard assignments since entropy is minimized by hard assignments.

3. Muon and Other Optimizers (14 pts)

In this question, we will focus on an idealized scenario where we are training a single linear layer with weight matrix W, and our batch consists entirely of copies of two datapoints whose individual gradients $\nabla_W L$ are:

$$\textbf{Gradient wrt. W for datapoint A}: \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \textbf{Gradient wrt. W for datapoint B}: \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (1)$$

Assume that the training batch consists entirely of a copies of datapoint A and b copies of datapoint B. The overall full-batch gradient is now[2]:

$$\textbf{Gradient}: \begin{bmatrix} a+b & a-b \\ a-b & a+b \end{bmatrix} \quad \text{which has SVD:} \quad \left(\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}\right)\begin{bmatrix} 2a & 0 \\ 0 & 2b \end{bmatrix}\left(\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}\right). \quad (2)$$

(a) (6pts) Assume $a = 1, b = 2$. **What is the resulting update ΔW when the gradient is used by gradient descent, SignGD, and Muon respectively?** Assume a learning rate of 1 and no momentum. Assume an idealized[3] Muon that takes the gradient and sets all its singular values to 1 instead of iterating through a matrix polynomial.

GD :_____

SignGD :_____

[1] Note: this loss is non-negative despite the seeming negative sign. Because this is a cheat-sheet-free exam, we tell you: the entropy of a probability distribution is always non-negative, is maximized by the uniform distribution, and is minimized by putting all the probability on a single possibility.

[2] For simplicity, we are assuming our loss sums over points in the batch instead of averaging over them.

[3] You can further ignore the matrix-shape-based terms in calculating the Muon update that allow the learning rate to be understood in terms of an RMS-Norm-to-RMS-Norm induced norm ball. This footnote exists because of the no clarifications, no questions policy. If you didn't have a question, you can ignore this footnote.

 All Rights Reserved. This may not be publicly shared without explicit permission.

Muon :_____

Solution: Just plugging in $a = 1, b = 2$ gives for the negative gradient (remember, we are doing gradient descent!):

$$\textbf{GD} : -\begin{bmatrix} 3 & -1 \\ -1 & 3 \end{bmatrix}$$

To see what SignGD would do with a learning rate of 1, we just just replace every entry with its sign:

$$\textbf{SignGD} : -\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

For Muon generally, we have to think about $\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}$ scaling to make sure that learning rate can be interpreted as a norm-ball that we are willing to move in the induced RMS-to-RMS norm. However, here, the matrix is square and so that scaling would be 1 anyway. We just have to take the SVD of the gradient and set its singular values to 1. In our case, that simplifies down to:

$$\textbf{Muon} : -\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

(b) (4pts) You increase your batch by inserting more copies of A and B at the same $1:2$ ratio. **Which of the three updates change from part (a)?**

☐ GD ☐ SignGD ☐ Muon

Solution: GD changes because in our model of losses adding together, the gradient also gets bigger with a bigger batch. However, because both of them normalize for size in some way, SignGD/Muon do not.

(c) (4pts) You now have a differently imbalanced training batch where $a = 10, b = 1$. **Which of the three updates above change from part (a)?**

☐ GD ☐ SignGD ☐ Muon

Solution: GD changes, SignGD changes. Muon does not. Why? Because with the new $a = 10, b = 1$, the gradient itself changes to $\begin{bmatrix} 11 & 9 \\ 9 & 11 \end{bmatrix}$ whose signs are $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$. However, because the two examples have orthogonal gradients, the change in their ratio doesn't change the singular vectors in this case. That means that the idealized Muon update also doesn't change.

This goes to illustrate the extent to which Muon inherently has more robustness to imbalance in a training dataset.

 All Rights Reserved. This may not be publicly shared without explicit permission.

4. Quantization Aware Training (14 pts)

Solution: Note: This quantization scheme is very closely related to the MXFP4 one implemented by OpenAI in their gpt-oss release this August. We merely changed some details to be easier to work with on the exam.

Recall from homework that linear quantization is defined by the goal of

$$r \approx S(q - Z) \tag{3}$$

where r is a floating point real number, q is a n-bit integer, Z is a n-bit integer, and S is a floating point real number. Z is the quantization zero point and S is quantization scaling factor. Both Z and S are quantization parameters that can be kept at higher precision because their cost will be amortized over lots of entries in a weight matrix being quantized together.

The actual quantization is done using rounding to the nearest integer:

$$q = \text{round}(\frac{r}{S} + Z). \tag{4}$$

Consider a 3×3 weight matrix whose values are represented in standard FP32 (float) format as

$$\begin{bmatrix} 0.25 & 2.0 & 1.5 \\ -1.0 & 0.5 & 0.25 \\ 0.4 & -1.0 & 0.5 \end{bmatrix}. \tag{5}$$

Now, you would like to use linear quantization to represent this weight matrix using 3 bits for q, so $q_{\min} = -4$ and $q_{\max} = 3$, and the eight possible values for q are: -4,-3,-2,-1,0,1,2,3. Let $r_{\min} = -1.0$ and $r_{\max} = 2.0$ be the smallest and largest values in the weight matrix. In order to use the full dynamic range of q, we should correspond $q_{\min}$ to $r_{\min}$ and $q_{\max}$ to $r_{\max}$.

(a) (9pts) What does the weight matrix (5) look like *after* applying linear quantization using the best computed S and Z values? Carefully look at the partially filled entries and **complete the empty entries in the Q matrix below:**

$$Q = \begin{bmatrix} \square & \square & 2 \\ -4 & -1 & -1 \\ \square & -4 & -1 \end{bmatrix} \tag{6}$$

(Hint: While you can solve for the blanks using explicit computation of the S and Z and applying (4), you can also get the values by inspection and thinking about what must happen in quantization.)

Solution: The answer is

$$Q = \begin{bmatrix} -1 & 3 & 2 \\ -4 & -1 & -1 \\ -1 & -4 & -1 \end{bmatrix}. \tag{7}$$

The 3 can be seen since it is the maximum value. Linear quantization has to map the same thing to the same quantized value and so the first -1 comes from a 0.25 which is already known to map to -1. The final -1 comes from the fact that if 0.5 quantizes to -1 and so does 0.25, then the piecewise constant nature of quantization means that everything in between does too.

 All Rights Reserved. This may not be publicly shared without explicit permission.

If we wanted to use the formula, $q = \text{round}\left(\frac{r}{S} + Z\right)$, we can find the scale S by using (3) to set up a system of two equations.

$$r_{\max} = S(q_{\max} - Z) \tag{8}$$
$$r_{\min} = S(q_{\min} - Z) \tag{9}$$

Subtracting these two equations, we have,

$$S = \frac{(r_{\max} - r_{\min})}{(q_{\max} - q_{\min})} \tag{10}$$
$$= \frac{3}{7}. \tag{11}$$

Once S is determined, we can compute the relevant zero-point Z.

$$Z = \text{round}(q_{\min} - r_{\min}/S) = \text{round}\left(-4 - \left(-\frac{7}{3}\right)\right) = -2. \tag{12}$$

Plugging in gives us the resulting values the long way.

$$\text{round}(\frac{0.25}{(\frac{3}{7})} - 2) = -1$$
$$\text{round}(\frac{2}{(\frac{3}{7})} - 2) = +3$$
$$\text{round}(\frac{0.4}{(\frac{3}{7})} - 2) = -1$$

(b) (5pts) Quantization-aware training quantizes the full-precision model weights w to w_q during each forward pass of training. It uses these quantized weights for computing activations. For backpropagation, we apply the "straight-through" estimator to compute the backpropped gradient through the quantization step to the underlying full-precision weights. This straight-through estimator treats the effect of quantization as though it were random external noise added to the full-precision weights. **Why do we do this?** (Select all that apply)

Solution: ☒ Quantization is either nondifferentiable (when viewed as a function into the integers) or has zero derivatives almost everywhere (when viewed as a function from $\mathbb{R} \to \mathbb{R}$) and would prevent meaningful updates if we actually differentiated it. **Solution:** This is the main issue for why we must do something like using the straight-through estimator once we have decided to do quantization during training.

☐ Because quantization saves memory during training when storing the momentum terms in the optimizer. **Solution:** This is not correct because the optimizer stores the momentum terms in full precision under our description of keeping parameters at full precision as we update them.

Solution: ☒ Because we want the final model to be used after quantization for inference, it is useful to allow training to learn how to reduce the loss due to quantization. **Solution:** This is the main reason why we decided to do quantization during training itself. If we quantize only afterwards, we

 All Rights Reserved. This may not be publicly shared without explicit permission.

get some loss from the quantization. However, we know that regularization techniques such as dropout as well as the reality of relative overparameterization means that there is a degree of at least approximate redundancy among the parameters. That means that there can be ways to get the same input-output behavior using different sets of weights — some of which might incur less quantization loss. Quantization-aware training is an attempt to encourage those more robust-to-quantization weights in the network.

5. Your Language Model is Secretly a Reward Model (24 pts)

In homework, you worked through the foundations of Direct Preference Optimization (DPO). If we optimize our policy $\pi_\theta(y|x)$ to maximize our reward model $r(x, y)$ (that tells us quantitatively how good y is as a response to x) while trying to stay close to a reference policy $\pi_{\text{ref}}(y|x)$ in KL Divergence:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(y|x)} \left[r(x, y)\right] - \beta\mathcal{D}_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$$

Then, the optimal solution takes the form:

$$\pi^*(y|x) = \frac{1}{\mathcal{Z}(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right) \tag{13}$$

where $\mathcal{Z}(x) = \sum_y \pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right)$ normalizes the probability and β is the hyperparameter that determines how much we want to stay close to the reference distribution π_{ref}. Here, x is the prompt/question and y is the response from the model.

(a) (6pts) **Manipulate (13) to express $r(x, y)$ in terms of $\pi^*(y|x)$, $\pi_{\text{ref}}(y|x)$, β, and $\mathcal{Z}(x)$.**

Solution: Starting with:

$$\pi^*(y|x) = \frac{1}{\mathcal{Z}(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x, y)\right)$$

Take the logarithm of both sides:

$$\log\pi^*(y|x) = -\log\mathcal{Z}(x) + \log\pi_{\text{ref}}(y|x) + \frac{1}{\beta}r(x, y)$$

Rearrange to solve for $r(x, y)$:

$$\frac{1}{\beta}r(x, y) = \log\left(\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)}\right) + \log\mathcal{Z}(x)$$

Multiply both sides by β:

$$r(x, y) = \beta\log\left(\frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)}\right) + \beta\log\mathcal{Z}(x)$$

(b) (4pts) Looking at your expression in the previous part, **is knowledge of $\mathcal{Z}(x)$ required to understand if the implicit reward prefers response y_1 to y_2 in response to prompt/question x?**

◯ Yes, we need $\mathcal{Z}(x)$

◯ No, we do not need $\mathcal{Z}(x)$

 All Rights Reserved. This may not be publicly shared without explicit permission.

Solution: We do not need $\mathcal{Z}(x)$ because it does not impact the difference $r(x, y_1) - r(x, y_2)$ because it contributes an identical additive offset to both. Since we can compute the difference, we can tell which one is bigger.

(c) (8pts) Now, consider the DPO gradient you derived in homework for training a network using a dataset consisting of triplets: two responses for each prompt x. The response that humans prefer is y_w and the response that humans don't prefer is y_ℓ.

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\beta \mathbb{E}_{(x,y_w,y_\ell)\sim\mathcal{D}} \left[\underbrace{\sigma\left(\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w)\right)}_{\text{weight}} \left[\underbrace{\nabla_\theta \log \pi_\theta(y_w|x) - \nabla_\theta \log \pi_\theta(y_\ell|x)}_{\text{update direction}} \right] \right]$$

where σ is the sigmoid function $\sigma(z) = 1/(1 + e^{-z})$, $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the implicit reward, and the distribution $\mathcal{D}$ just represents what SGD is doing.

(i) Consider a case where the model is extremely confident that the dispreferred response y_ℓ is actually better than the preferred response y_w (i.e., $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$). **What happens to the weight term in the gradient, i.e.** $\sigma(\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w))$**?** Select one response.

◯ The gradient weight becomes substantial, approaching 1 if $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$.

◯ The gradient weight approaches zero if $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$.

◯ The gradient weight approaches $\frac{1}{2}$ if $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$.

Solution: The gradient update weight becomes substantial when there is a disagreement between the model and the human raters.
If $\hat{r}_\theta(x, y_\ell) \gg \hat{r}_\theta(x, y_w)$, then $\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w)$ is a large positive number. Since the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}} \to 1$ as $z \to \infty$, the weight approaches 1 (maximum strength).
Conversely, if the model already correctly ranks the outputs ($\hat{r}_\theta(x, y_w) \gg \hat{r}_\theta(x, y_\ell)$), then $\hat{r}_\theta(x, y_\ell) - \hat{r}_\theta(x, y_w)$ is a large negative number, so $\sigma(\text{large negative}) \approx 0$. The gradient weight becomes nearly zero, and the model stops updating on this example — as long as the batch contains other examples that have higher weights. Things can get tricky if everything has a low weight when using adaptive optimizers.

(ii) Why is the adaptive-weighting in DPO desirable compared to the standard supervised fine-tuning (SFT) approach that simply maximizes $\mathbb{E}_{(x,y_w)\sim\mathcal{D}} \log\left(\pi_\theta(y_w|x)\right)$? (**Mark all that apply**)

☐ Our preference training data just tells us which of two responses were better. It doesn't tell us if both were somewhat bad or both were good but one was better. And yet, standard SFT would treat all the preferred responses as though they were gold-standard perfect responses and completely ignore the information present in knowing which alternatives are less preferred.

Solution: This is a correct response. For every preference pair (x, y_w, y_ℓ), SFT would simply maximize $\log\left(\pi_\theta(y_w|x)\right)$ with uniform weight, regardless of whether the model already prefers y_w over y_ℓ. The very rough analogy for SFT would be to doing binary classification using only squared loss and only the positive examples. Meanwhile, DPO is very roughly analogous to using logistic loss with both positive and negative examples.

☐ Because standard SFT would apply the same amount of update towards winners even when they were already winning, it is more prone to triggering catastrophic forgetting in the model during a fine-tune, especially as we fine-tune for multiple epochs. The DPO updates are comparatively more minimal.

Solution: This is also a correct response.

 All Rights Reserved. This may not be publicly shared without explicit permission.

- ☐ Attenuating the strength of the gradients in this DPO way helps prevent numerical overflow as compared to standard SFT.

  Solution: This is an incorrect response: multiplying by numbers that are moderate sized like this is not enough to defend against overflow.
- ☐ Doing the DPO update better prevents vanishing gradients.

  Solution: This is also incorrect. The issue of vanishing gradients is orthogonal. DPO updates neither help nor hurt.

(d) (6pts) Imagine you are full fine-tuning a 7B instruction-following model using DPO with β selected from the low end of the typical range and reference policy $\pi_{\text{ref}} = \pi_{\text{SFT}}$. After several epochs of training on preference pairs, you observe:

- Training loss: decreased significantly from initial values
- KL divergence $\mathcal{D}_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$: has grown substantially larger than typically recommended
- Generated responses: contain repetitive phrases and degenerate patterns. They have gotten worse than what π_{SFT} generated.
- Held-out preference accuracy: very high (model correctly ranks test preferences better than π_{SFT} did)
- Human evaluation on new prompts: responses are often incoherent or low-quality

Below are potential strategies for addressing the training issues identified above. **Select ALL strategies that would likely be helpful.**

- ☐ Increase β substantially (e.g., by a factor of 3-5x) to strengthen the KL penalty and stay closer to the reference policy.
- ☐ Decrease β to allow the model more freedom to optimize the preference objective.
- ☐ Implement early stopping.
- ☐ Continue training for additional epochs to allow the model to find better solutions.
- ☐ Use DPO with a LoRA fine-tune that trains fewer learnable parameters instead of a full fine-tune.

Solution: The model has suffered from **mode collapse** or **reward overoptimization**. The root cause is that the model has overfit to just evaluating reward.

Consider the RLHF objective:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(y|x)}\left[r(x,y)\right] - \beta\mathcal{D}_{\text{KL}}[\pi_\theta(y|x)\|\pi_{\text{ref}}(y|x)]$$

The parameter β controls the strength of the KL penalty term, which keeps the policy close to the reference policy. When β is too small:

- The KL penalty is weak relative to reward terms
- Reward maximization dominates over the constraint
- The policy can drift very far from π_{ref}, resulting in high KL divergence

The model discovered that being prone to generating repetitive or degenerate patterns is compatible with increased preference accuracy on the training distribution but result in incoherent outputs.

Explanation of correct strategies:

Increasing β substantially (e.g., by a factor of 3-5x): The current β value is too small, resulting in a weak KL penalty term. This allows the policy to drift excessively far from the reference policy, as evidenced by the high KL divergence. Increasing β strengthens the regularization, constraining the policy to remain in a more reasonable region of policy space where outputs hopefully stay coherent.

 All Rights Reserved. This may not be publicly shared without explicit permission.

Implementing early stopping: Setting a maximum acceptable KL threshold and stopping training when it's exceeded serves as a safety mechanism. This prevents the model from entering the degenerate high-KL regime even if other hyperparameters aren't perfectly tuned. The model can be restored to the best checkpoint based on validation metrics.

Use DPO with a LoRA fine-tune: If the underlying challenge is a kind of overfitting (to the reward estimation problem) manifesting as catastrophic forgetting (of how to actually generate good generations), doing a LoRA fine-tune drastically reduces the number of parameters being adjusted and thereby has a regularizing effect.

Why other options are incorrect:

Decrease β: This is the opposite of what's needed. β is already too low, causing insufficient regularization. Decreasing it further would worsen the problem by allowing even more drift from the reference policy.

Continue training for additional epochs: The model has already overoptimized on the training objective, achieving very high preference accuracy while producing incoherent outputs. Additional training would likely increase KL divergence further and worsen the mode collapse rather than helping the model escape it. There is nothing in the preference-ordering training data that would fight this.

6. Using Redundancy in Parameterization (15 pts)

Assume you are given the following dense layer:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \tag{14}$$

with the shape of the weights W being $d_{\text{out}} \times d_{\text{in}}$ and the biases $\mathbf{b}$ being d_{out}-dimensional. There is an iid random Gaussian initialization of:

$$W[i,j] \sim N(0, \frac{1}{d_{\text{in}}}) \qquad b[i] \sim N(0, 10^{-4}) \tag{15}$$

for i, j ranging over the shape of the weight matrix and bias vector. Here, the first argument to N is the mean and the second argument is the variance.

You are dealing with a particularly annoying codebase in which you cannot adjust the Adam optimizer being used or change any of the optimizer parameters, including the learning rate, momentum terms, etc.[4] Feel free to think about the SignGD optimizer as standing-in for Adam in this problem.

However, you are able to change the network definition in (14) as well as the initialization described by (15). **How can you adjust these definitions in (14) and (15) so that the input-output behavior training trajectory matches what would happen if you had used a 10x higher learning rate for Adam in the original network (14) and initialization (15)?**

$\mathbf{y} =$ _____

$W[i,j] \sim$ _____

[4] To avoid technicalities, assume that the ϵ parameter in Adam that prevents division by zero is just $\epsilon = 0$. You can ignore this footnote if you weren't worried about that part — this exists only because this is a zero-questions, zero-clarifications exam.

 All Rights Reserved. This may not be publicly shared without explicit permission.

$$b[i] \sim _____$$

Solution:

The key insight is to realize that Adam, like signSGD, uses the learning rate to determine the size of steps taken in absolute terms. So a step size of $\eta = 0.001$ can be caricatured as making each step change parameters in the third decimal digit. If we want the effect of a ten times higher learning rate, we would ordinarily need to change those same parameters in the second decimal digit each time. But we can't change the learning rate. So, we need to change the parameters so that the relevant information in the second decimal digit is now found in the third decimal digit — in other words, we need to make all the parameters ten times smaller.

Of course, if we just did that, we would have a very different network from an input-output point of view. So, we need to compensate for our 10-times smaller parameters somehow from an input-output point of view. To do that, we can just add a factor of ten into the equation for $\mathbf{y}$. This gives the following answer:

$$\mathbf{y} = 10(W\mathbf{x} + \mathbf{b}) \tag{16}$$

$$W \sim N(0, \frac{1}{10^2 d_{\text{in}}}) \tag{17}$$

$$b \sim N(0, 10^{-6}) \tag{18}$$

$$\tag{19}$$

Where the final realization required to see the answer is that to make a zero-mean Gaussian ten times smaller, you need to make its variance a hundred times smaller.

7. Debugging Generative Models (12 pts)

You are training your first generative model, and you're very excited as your training and validation losses have both dropped to nearly zero. But when you try to generate some samples, only garbage comes out! What is wrong or missing in each of the following implementations, and/or how would you fix it? **Answer briefly. You don't need to give the exact fix in code, just tell us what you need to do.** If you say X is missing, we will understand that the implied fix is to include X.

(a) (6pts)

```
class CausalAttention(nn.Module):
    def forward(self, x):
        Q = self.W_q(x)   # (batch_size, seq_len, d_model)
        K = self.W_k(x)     # (batch_size, seq_len, d_model)
        V = self.W_v(x)   # (batch_size, seq_len, d_model)
        scores = torch.matmul(Q, K.transpose(-2, -1))
        scores = scores / math.sqrt(self.d_model)
        attention_weights = F.softmax(scores, dim=-1)
        attn_output = torch.matmul(attention_weights, V)
        output = self.W_o(attn_output)
        return output
```

 All Rights Reserved. This may not be publicly shared without explicit permission.

Above, we have what should be a causal attention block used as part of a standard Transformer language model recipe for a GPT-style model pretrained using batches consisting of teacher-forced entire sequences and next-token prediction loss. To sample from the model, we would generate tokens autoregressively. **What is wrong with the above implementation for training?**

Solution: The attention weights are not masked, so the model can attend to future tokens. Since the next token is actually very good at predicting the next token, the model might learn that. However, when used to do generation autoregressively, that next token is not present. So we will get bad generations.

To solve the problem, we need to mask the attention scores by setting scores corresponding to looking into the future to $-\infty$.

(b) (6pts)

```
class VariationalAutoEncoder(nn.Module):
    def get_loss(self, x):
        # Encode x.
        z_mean, z_log_std = self.encoder(x)

        # Add noise to z.
        noise = torch.randn(z_mean.shape)
        z = z_mean + torch.exp(z_log_std) * noise

        # Decode z back to x, with mean-squared reconstruction loss.
        x_pred = self.decoder(z)
        loss = F.mse_loss(x, x_pred)
        return loss
```

Above, we have what should be the entire loss function for training a variational auto-encoder. To sample from this model, we would sample $z \sim N(0, 1)$, and decode with our learned decoder. **What is wrong with the above implementation for training?**

Solution: The VAE auxiliary loss — the KL divergence term between the approximate posterior and the desired prior for sampling — is missing from the loss function.

8. Rank of Updates (24 pts)

Throughout this problem, assume a learning rate of $\eta = 1$ and simple SGD with no momentum is being used to do weight updates.

(a) (6pts) Let $\mathbf{y} = W\mathbf{x}$ represent a fully connected layer where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input to the layer, and $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$ is the output of the layer.

This layer is being trained with a minibatch consisting of a single input, and backprop from the final loss brings a nonzero upstream gradient $\mathbf{g} = \nabla_{\mathbf{y}} L$ where L is the final loss in our deep network. Note that $\mathbf{g}$ has the same shape as $\mathbf{y}$ since it is a gradient.

Recall that: $\frac{\partial L}{\partial W[i,j]} = x[j] \frac{\partial L}{\partial y[i]}$ for $\mathbf{y} = W\mathbf{x}$.

What is the SGD update of the weight matrix, ie ΔW? (Your answer should be in terms of $\mathbf{x}, \mathbf{g}$.)

$$\Delta W = \underline{\qquad\qquad\qquad\qquad}$$
$$\text{rank}(\Delta W) = 1$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

Solution: Since we only saw a single training example, the input x is a vector of dimension d_{in} and the upstream gradient $\mathbf{g}$ is a vector of dimension d_{out}. Following the formula for $\frac{\partial L}{\partial W[i,j]}$, the gradient on the weight matrix is the outer product $\mathbf{g}\mathbf{x}^\top$ since that matches the shape as we look at all i, j together. The simple GD update ΔW is thus the negative outer product $\Delta W = -gx^T$ with a minus sign because we are doing gradient descent and with no constant in front because the learning rate is 1.

Therefore, the update ΔW has rank 1 because it is the outer product of two vectors.

(b) (6pts) Now, assume we process a batch of B training samples at a time. That is, our input is now a matrix $X = [\mathbf{x}_1\ \mathbf{x}_2\ \cdots\ \mathbf{x}_B]$ with shape $d_{\text{in}} \times B$ and our upstream gradient (from backprop) is now a corresponding matrix G with shape $d_{\text{out}} \times B$ where the i-th column is $\nabla_{\mathbf{y}_i} L_i$ the gradient of the loss with respect to the layer's output with input $\mathbf{x}_i$. Assume that $B < \min(d_{\text{in}}, d_{\text{out}})$. **What is the new update of the weight matrix ΔW?**

$$\Delta W = _____$$

Solution: The update ΔW is given by the matrix multiplication $\Delta W = -GX^\top$.

Why, because the impact of the batch is add together the gradients from each of the individual examples in the batch. That is exactly what the outer-product view of matrix multiplication tells us is happening in $GX^\top$.

Here, if we are doing the standard thing in deep learning, we would average the gradient over the batch and hence our update would be $\Delta W = -\frac{1}{B} GX^\top$. **What is the rank of this update?** Assume the inputs in the batch are linearly independent and the upstream gradient also has full rank.

$$\text{rank}(\Delta W) = _____$$

Solution: The rank of the update ΔW is determined by the ranks of G and X. Since both matrices have full rank, the rank of the update ΔW is $\min(B, d_{\text{in}}, d_{\text{out}})$. In the problem, batch size B is the smallest of these three values and so the answer is B.

(c) (6pts) Now, consider a convolutional layer with C_{in} input channels and C_{out} output channels, where each convolutional filter has spatial size $K \times K$. We will use same padding and a stride of 1 so that if the layer's input has size $W \times H \times C_{\text{in}}$, then the layer's output has size $W \times H \times C_{\text{out}}$.

Recall that such a convolutional layer can be represented as a linear layer, wherein all the filters are flattened together into a single matrix F and the input image is reshaped/recast into a matrix X so that each column of X corresponds to all the C_{in} input channels in a flattened $K \times K$ patch of the input image. **Fill in the following blanks:**

i. The reshaped filter F has matrix shape $C_{\text{out}} \times \underline{K \cdot K \cdot C_{\text{in}}}$.

ii. The output $Y = FX$ of this operation has shape **Solution:** $\underline{C_{\text{out}}}$ $\times \underline{W \cdot H}$. *(Hint: The output can be reshaped into an image of shape $W \times H \times C_{out}$. Make sure your answer is consistent!)*

iii. To be able to properly represent this convolutional layer, the input X should have shape: **Solution:** $\underline{K \cdot K \cdot C_{\text{in}}}$ $\times$ **Solution:** $\underline{W \cdot H}$.

 All Rights Reserved. This may not be publicly shared without explicit permission.

Solution: In the problem above, we remember that all linear operations on finite dimensional spaces can be represented as a matrix. This is the same thing that we do when we want to use Muon with a convolutional layer.

(d) (6pts) Building on the previous part where $Y = FX$, assume we are updating the weights of this convolutional layer after seeing a single training image. **What is the update of the (flattened) weight matrix ΔF in terms of X, which is the reshaped input image, and upstream gradient $G = \nabla_Y L$?** (The upstream gradient has also been shaped to match the operation output Y above.)

$$\Delta F = _____$$

Even though we have only a single training image, how big can the rank of the update matrix ΔF be? Your answer should be in terms of $K, C_{\text{in}}, C_{\text{out}}, W, H$.

$$\text{rank}(\Delta F) \leq _____$$

Briefly explain why this rank can be strictly bigger than 1 while in part (a), the rank is 1 when there's a single training example in the batch.

Solution: Note that from the previous part, the reshaped input X has shape $(K \cdot K \cdot C_{\text{in}}) \times (W \cdot H)$ and the upstream gradient G should have shape equal to the flattened layer output, which is $C_{\text{out}} \times (W \cdot H)$. Additionally, note that the update ΔF has the same shape as the flattened filter, which is $C_{\text{out}} \times (K \cdot K \cdot C_{\text{in}})$.

The gradient is then $G \cdot X^\top$ and so the gradient descent update with learning rate 1 is just $\Delta F = -G \cdot X^\top$. Note that due to the weight sharing property of CNNs, for a single training input, X is no longer a vector, but a matrix where each column contains all the patches of the image that a given filter "sees". This matrix view is a different perspective on this exact issue which you saw in your earlier homework problem (4.7) of directly looking at backprop in the context of a convolutional filter.

While for traditional batches, it is conventional to average over the batch when looking at the loss, note that the summing that is happening here is not typically averaged over. This summing is a consequence of weight sharing and the gradients on these shared weights need to get added up. That's why there is no second correct option in this case.

The rank of the update is therefore $\min(C_{\text{out}}, K \cdot K \cdot C_{\text{in}}, W \cdot H)$.

In particular, each entry in the input image is represented multiple times in this X matrix. Because images have interesting regions, the rank of this X matrix tends to be significant and similarly, because these different regions actually end up mattering differently in later layers, the gradient coming downstream is also different in different places and so that matrix also has significant rank. This reality of different patches in the image being different and the weight-sharing in a CNN is why this rank of the update tends to be strictly bigger than 1.

9. muP (24 pts)

The maximal update parameterization (muP) provides a helpful scaling law for learning rates, such that training dynamics remain similar as we adjust the size of our neural network. Throughout this problem, assume we are utilizing an off-the-shelf SignGD optimizer (standing in as an idealization of Adam) with a minibatch size of 1.

 All Rights Reserved. This may not be publicly shared without explicit permission.

(a) (8pts) Consider a bias-free dense linear layer defined as $\mathbf{y} = W\mathbf{x}$, where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Assume $\mathbf{x}$ has RMS Norm 1. **How should we scale the learning rate for W so that the RMS-norm change in $\mathbf{y}$ remains constant regardless of the shape of W?**

(Hint: Remember part (a) of Problem 10: "Rank of Updates" to help.)

Select one:

◯ $O(1)$ i.e. constant	◯ $O(d_{\text{in}}d_{\text{out}})$	◯ $O(\frac{d_{\text{in}}}{d_{\text{out}}})$
◯ $O(\frac{1}{\sqrt{d_{\text{in}}}})$	◯ $O(\frac{1}{\sqrt{d_{\text{out}}}})$	◯ $O(\sqrt{\frac{2}{d_{\text{in}}+d_{\text{out}}}})$
◯ $O(\frac{1}{d_{\text{in}}})$	◯ $O(\frac{1}{d_{\text{out}}})$	◯ $O(\frac{2}{d_{\text{in}}+d_{\text{out}}})$
◯ $O(\sqrt{d_{\text{in}}})$	◯ $O(\sqrt{d_{\text{out}}})$	◯ $O(\sqrt{\frac{d_{\text{in}}+d_{\text{out}}}{2}})$
◯ $O(d_{\text{in}})$	◯ $O(d_{\text{out}})$	◯ $O(\frac{d_{\text{in}}+d_{\text{out}}}{2})$

Solution: The learning rate should scale as $\frac{1}{d_{\text{in}}}$.

You saw this in lecture and homework, but to work it out from scratch: We have a rank-1 gradient which is $\mathbf{g}\mathbf{x}^\top$ if $\mathbf{x}$ is the input and $\mathbf{g} = \nabla_{\mathbf{y}} L$ is the backpropagated gradient of the loss. Because of the nature of outerproduct and the fact that $\text{sign}(ab) = \text{sign}(a) \cdot \text{sign}(b)$ for scalars, we know $\text{sign}(\mathbf{g}\mathbf{x}^\top) = \text{sign}(\mathbf{g})(\text{sign}(\mathbf{x}))^\top)$. Since this is a rank-1 matrix that is filled with ± 1, we know that its spectral norm squared is equal to its Frobenius norm squared which is equal to $d_{\text{in}}d_{\text{out}}$. Since the RMS Norm 1 vector $\mathbf{x}$ has a positive inner product with its own $\text{sign}(\mathbf{x})$, namely $\|\mathbf{x}\|_1$, we know that the update $\Delta W = -\eta \text{sign}(\mathbf{g})(\text{sign}(\mathbf{x}))^\top$ will actually result in a $\mathbf{y}$ update of:

$$\begin{aligned}\Delta W\mathbf{x} &= -\eta \text{sign}(\mathbf{g})(\text{sign}(\mathbf{x}))^\top \mathbf{x} \\ &= -\eta \|\mathbf{x}\|_1 \text{sign}(\mathbf{g})\end{aligned}$$

Generically, the 1 norm of an RMS-Norm-1 vector is going to scale with the number of dimensions in this vector, namely d_{in} because we are generically adding together d_{in} positive numbers that are all in the neighborhood of 1 (on average).

To have the update have constant RMS-Norm, we note that $\text{sign}(\mathbf{g})$ has RMS-Norm 1 by construction and so we need need the learning rate η to scale as $O(\frac{1}{d_{\text{in}}})$.

(b) (8pts) Let us now consider embedding layers of a language model, parameterized by a matrix $E \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. Recall that performing lookup on an embedding layer can be understood as multiplying by a *one-hot* vector of dimension d_{in}. **How should we scale the learning rate for the embedding layer in terms of its shape to ensure that the RMS-norm change in the output of the embedding layer remains constant regardless of d_{in} and d_{out}.**

Select one:

◯ $O(1)$ i.e. constant	◯ $O(d_{\text{in}}d_{\text{out}})$	◯ $O(\frac{d_{\text{in}}}{d_{\text{out}}})$
◯ $O(\frac{1}{\sqrt{d_{\text{in}}}})$	◯ $O(\frac{1}{\sqrt{d_{\text{out}}}})$	◯ $O(\sqrt{\frac{2}{d_{\text{in}}+d_{\text{out}}}})$
◯ $O(\frac{1}{d_{\text{in}}})$	◯ $O(\frac{1}{d_{\text{out}}})$	◯ $O(\frac{2}{d_{\text{in}}+d_{\text{out}}})$
◯ $O(\sqrt{d_{\text{in}}})$	◯ $O(\sqrt{d_{\text{out}}})$	◯ $O(\sqrt{\frac{d_{\text{in}}+d_{\text{out}}}{2}})$
◯ $O(d_{\text{in}})$	◯ $O(d_{\text{out}})$	◯ $O(\frac{d_{\text{in}}+d_{\text{out}}}{2})$

Solution: The learning rate should be constant.

 All Rights Reserved. This may not be publicly shared without explicit permission.

Why? Because d_{in} is effectively 1 because of the one-hot nature of the inputs. All that matters is that the RMS-Norm of the update to the relvant column of the E matrix be constant. However, SignGD will achieve that with a constant learning rate.

(c) (8pts) We will now consider learnable bias vectors. Consider a dense layer defined as $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$. Again, assume that $\mathbf{x}$ has RMSNorm 1.

i. Suppose we did *not* use muP scaling – i.e. we use the same learning rate for all parameters, regardless of shape – **how much does an update to the bias vector $\Delta\mathbf{b}$ contribute to the corresponding expected RMS-norm-squared of the change in $\mathbf{y}$, in comparison to the update to the weight matrix ΔW?** Express your answer as a rough proportion contribution-from-bias : contribution-from-dense-weight-matrix.
(Hint: For simplicity, think of the input $\mathbf{x}$ as being a vector entirely of 1s and the gradient $\nabla_{\mathbf{y}} L$ coming from upstream as also a vector consisting entirely of 1s.)
Select one:

◯ $1:1$	i.e. same contribution	◯ $1:2$	i.e. Bias contributes half as much
◯ $1:d_{\text{in}}$		◯ $1:d_{\text{out}}$	
◯ $1:d_{\text{in}}^2$		◯ $1:d_{\text{out}}^2$	
◯ $d_{\text{in}}:1$		◯ $d_{\text{out}}:1$	
◯ $d_{\text{in}}^2:1$		◯ $d_{\text{out}}^2:1$	

Solution: The proportion is $1 : d_{\text{in}}$. This can be understood by considering the shapes – for each output feature, the bias contributes a change of 1, and the dense parameters contributes a change of d_{in}.

ii. One option for applying muP to learnable biases is to treat biases as matrices with a shape of $d_{\text{out}} \times 1$, i.e. thinking of their $d_{\text{in}} = 1$ as though they were weights multiplying a constant feature whose value is always 1. We can then apply the same muP learning-rate scaling rule derived in part (a) of this problem for the learnable biases, along with applying the muP scaling for the weights based on their shape. **What is the resulting rough proportion of contribution-from-bias : contribution-from-dense-weight-matrix** for the resulting updates?
Below, d_{in} refers to the d_{in} of the learnable dense weight matrix. The d_{out} is the same for both the biases and the weight matrix.
Select one:

◯ $1:1$	i.e. same contribution	◯ $1:2$	i.e. Bias contributes half as much
◯ $1:d_{\text{in}}$		◯ $1:d_{\text{out}}$	
◯ $1:d_{\text{in}}^2$		◯ $1:d_{\text{out}}^2$	
◯ $d_{\text{in}}:1$		◯ $d_{\text{out}}:1$	
◯ $d_{\text{in}}^2:1$		◯ $d_{\text{out}}^2:1$	

Solution: The ratio between bias-dense will now be 1:1 because their different learning rates will make each of them contribute a change of the same order.
In practice, this is often a bad idea despite it being reasonable in theory because it effectively makes the learning rate for the biases too big for small to moderate sized batches. A useful strategy is to keep the learning rate for the weights and biases in an individual layer the same, while using muP scaling to adjust those learning rates across different layers.

10. Comparing Sequence Model Architectures: Compute (20 pts)

In this question, you will compare three major sequence model architectures: classic Recurrent Neural Networks (RNNs), Transformers, and modern State Space Models (SSMs). Let T be the sequence length and d be the embedding dimension. For State Space Models, let N be the state-expansion — i.e. by what factor (typically a number like 64) the state is bigger than the embedding dimension.

 All Rights Reserved. This may not be publicly shared without explicit permission.

Assume that the cost of a single floating-point operation (FLOP) is constant. **Fill in the missing cells of the below table of runtimes (in big-O notation), and provide a brief explanation in the corresponding section on the next page.** You can assume that $T \gg d \gg N$.

Task	RNNs	Transformers	SSMs (S4D style)
Training cost	$O(Td^2)$ due to both RNN forward pass and backprop using T recurrent updates using matrices with dimension d	**(a) Solution:** $O(Td(T+d))$ for time to compute dot-product attention and all the Q, K, V projections. Given the assumptions, the dominant term is $O(T^2d)$.	**(b) Solution:** $O(Td((\log T)+d+N))$ for SSM convolution using the FFT, computing all the inputs/outputs of the state-space-block, and materializing all d of the length T convolutional filters. Given the assumptions, we can't know if $\log T$ or d is the dominant term, but the N can be dropped.
Training parallelization	Poor due to inherently sequential computation due to the nonlinearities encountered along the time axis	Excellent (every token's attention computation can be fully-independent)	Good (FFT can be parallel with depth $\log T$, but not as good as self-attention because of how GPUs are optimized.).
Inference cost for auto-regressive generation of a sequence of T tokens	**(c) Solution:** $O(Td^2)$ due to T updates of the hidden state, each taking $O(d^2)$ time	**(d) Solution:** $O(T^2d)$ for sum of linearly increasing costs. The cost of t-th token is $O(td+d^2)$ for attention-score calculation across all previous tokens, assuming K, V for past steps are stored in the kv-cache.	$O(Td(d+N))$ for T steps of cost $O(d^2+dN)$ due to an update of the state with dimension dN that breaks down into $O(d^2)$ to compute inputs/outputs and $O(dN)$ to do the independent scalar state updates. Under our assumptions, the cost simplifies to $O(Td^2)$.

Because this is a cheat-sheet free exam, we provide you with some S4D basics here. S4D is the style you saw in lecture before Mamba-style models with selective state spaces.

- In modern SSMs, the state-space block performs a role analogous to that of a causal multi-head-attention block in Transformer architectures.
- In S4D models, the state-space part has diagonal learnable dynamics and the operation of the state-space block is to perform a different learnable linear-time-invariant scalar convolution for each embedding dimension.
- It is possible to speed up the computation of a full-length convolution of a scalar sequence of length n with another scalar sequence of length n to happen in $O(n \log n)$ time using FFT-based tricks.
- It is possible to directly materialize a single scalar impulse response of length T using $O(NT)$ computation but only $O(N+T)$ memory.
- As is the case in multi-head-attention for computing the queries, keys, and values, $O(d^2T)$ computations are needed to compute all the inputs and outputs to S4D's core state-space-model. The counterparts to fully-connected feedforward MLPs also cost the same.

Brief Explanations For Your Answers:

 All Rights Reserved. This may not be publicly shared without explicit permission.

(a) (2pts)

(b) (2pts)

(c) (2pts)

(d) (2pts)

11. Residual Connections with PreNorms (20 pts)

The deeper a neural network grows, the more likely it will suffer from either vanishing gradients or exploding gradients due to the multiplicative nature of the chain rule.

Fig 1 shows a simplified example of a residual block with layer normalization in the now default "pre-norm" style.

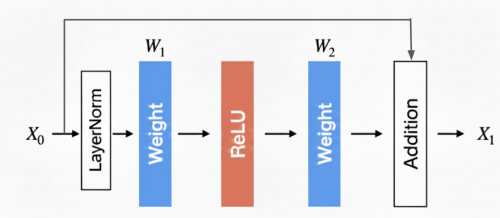


Figure 1: Diagram for a Residual Block with LayerNorm

(a) (3pts) Assuming that we have zero biases, **what is the expression for X_1 in terms of W_1, W_2 and X_0?** Assume $W_i \in \mathbb{R}^{n\times n}$, $X_i \in \mathbb{R}^n$. You can just call both LayerNorm and component-wise ReLU as functions here, you don't need to expand them.

Solution: The input X_0 passes through LayerNorm, then W_1, then ReLU, and then W_2. This entire result is added back to X_0.

$$X_1 = W_2\text{ReLU}\left(W_1\text{LayerNorm}(X_0)\right) + X_0$$

.

(b) (5pts) **Compute the Jacobian** $\frac{\partial X_1}{\partial X_0}$. Express your answer in terms of W_1, W_2, the Jacobian of LayerNorm, J, and the Jacobian of the ReLU given by diagonal matrix D whose i-th term on the diagonal is 1 if the corresponding ReLU is on, and 0 if it is off.

Based on the structure of your result, **explain why the residual connection helps prevent vanishing gradients during backprop for a deep network**.

Solution: The expression for X_1 is $X_1 = W_2\text{ReLU}(W_1 Z) + X_0$.

Applying the linearity of differentiation:

$$\frac{\partial X_1}{\partial X_0} = \frac{\partial\left(W_2\text{ReLU}(W_1 Z)\right)}{\partial X_0} + \frac{\partial X_0}{\partial X_0}$$

The second term is the identity matrix, I. For the first term, we apply the chain rule multiple times:

$$\frac{\partial\left(W_2\text{ReLU}(X_1 Z)\right)}{\partial X_0} = W_2\frac{\partial\text{ReLU}(W_1 Z)}{\partial W_1 Z}W_1 J$$

We recognize $D = \frac{\partial\text{ReLU}(X_1 Z)}{\partial X_1 Z}$ (the diagonal matrix of ReLU derivatives defined in the problem).

 All Rights Reserved. This may not be publicly shared without explicit permission.

The final expression for the Jacobian is:

$$\frac{\partial X_1}{\partial X_0} = W_2 D W_1 J + I$$

The residual connection preserves gradient norms better because the Jacobian contains the identity I. The presence of I ensures that even if the function term $W_2 D W_1 J$ is close to the zero matrix, the overall Jacobian is not vanishing. When we multiply together many such terms, the product of the identity matrices will always give at least one identity term — this represents the "gradient superhighway" that residual connections enable.

(c) (12pts) For simplicity, instead of LayerNorm, let us consider an RMSNorm layer with no learnable parameters that scales its input vector so that the output vector points in the same direction, but has RMSNorm 1.

If the input is the $d-$dimensional vector $\mathbf{X}$, the output vector $\mathbf{Z} = \sqrt{d}\frac{\mathbf{X}}{\|\mathbf{X}\|_2}$.

When backpropping the gradient through the RMSNorm Layer, what is the applied Jacobian $J = \frac{\partial Z}{\partial X}$?

(Hint: while it is completely possible to grind the calculation out using vector calculus, it might be useful to think about this Jacobian J using what the derivative means conceptually. Let $\mathbf{R}(\mathbf{X})$ be the RMSNorm Layer acting on $\mathbf{X}$. Then $\mathbf{R}(\mathbf{X} + \mathbf{\Delta X}) \approx \mathbf{R}(\mathbf{X}) + J\mathbf{\Delta X} + O(\|\mathbf{\Delta X}\|^2)$.

To understand J, consider a neighborhood of $\mathbf{X}$ using a local orthogonal coordinate system. In other words, use a convenient coordinate system for $\mathbf{\Delta X}$. Let the first basis vector be aligned to $\mathbf{X}$ itself and have the rest be $\mathbf{h}$ that are orthogonal to it. Use the linearity of the derivative to understand those different directions separately.

What is the impact on the output of an RMSNorm Layer of growing/shrinking $\mathbf{X}$ by adding something aligned to $\mathbf{X}$?

What is the impact on the output of an RMSNorm Layer of adding a tiny $\mathbf{h}$ that is orthogonal to $\mathbf{X}$? — You might find it helpful to think about the Pythagorean theorem and the fact that $\sqrt{1+\Delta^2} \approx 1 + \frac{\Delta^2}{2}$ when Δ is small.

Once you understand what the matrix J does to all basis vectors, what must it be?)

Vector Calculus Facts from cheatsheat:

Derivative of a norm-squared: $\frac{\partial}{\partial \mathbf{x}} \mathbf{x}^\top \mathbf{x} = 2\mathbf{x}^\top$

Scalar-vector product rule: $\frac{\partial}{\partial \mathbf{x}}(f(\mathbf{x})\mathbf{g}(\mathbf{x})) = f(\mathbf{x})\frac{\partial}{\partial \mathbf{x}}(\mathbf{g}(\mathbf{x})) + \mathbf{g}(\mathbf{x})\frac{\partial}{\partial \mathbf{x}}(f(\mathbf{x}))$

This plus scalar calculus should suffice to get the answer.

Solution: We want the RMS Norm to be 1 and so the Euclidean Norm-squared must be d. This means $\mathbf{Z} = \sqrt{d}\frac{\mathbf{X}}{\|\mathbf{X}\|_2}$. We compute a unit vector in the direction of $\mathbf{X}$ and then scale it up to have the desired RMS Norm.

For the Jacobian, we first notice that this must be a linear function by the definition of a derivative. The impact of a ΔX purely in the direction of $\mathbf{X}$ is to do nothing since that changes the scale of the input, not its direction. Consequently, the Jacobian J must null out that direction completely. A small change in an orthogonal direction does not impact the length of the vector to first order because of the Pythogorean calculation given in the hint. Consequently, the RMSNorm operation just acts like multiplication by $\frac{\sqrt{d}}{\|\mathbf{X}\|_2}$ to a small change in that direction. This is a scaled identity. Consequently, the Jacobian is $\frac{\sqrt{d}}{\|\mathbf{X}\|_2}(I - \frac{1}{d}\mathbf{Z}\mathbf{Z}^\top)$ since one needs to project away anything in the direction of $\mathbf{X}$ and otherwise just scale.

 All Rights Reserved. This may not be publicly shared without explicit permission.

This can also be seen using vector calculus directly:

$$\frac{\partial Z}{\partial X} = \frac{\partial}{\partial X}(\sqrt{d}\frac{\mathbf{X}}{\|\mathbf{X}\|_2}) \tag{20}$$

$$= \sqrt{d}(\frac{1}{\|\mathbf{X}\|_2}I + \mathbf{X}\frac{\partial}{\partial X}(\frac{1}{\|\mathbf{X}\|_2})) \tag{21}$$

$$= \sqrt{d}(\frac{1}{\|\mathbf{X}\|_2}I - \mathbf{X}(\frac{1}{\|\mathbf{X}\|_2^2}\frac{\partial}{\partial X}(\|\mathbf{X}\|_2))) \tag{22}$$

$$= \sqrt{d}(\frac{1}{\|\mathbf{X}\|_2}I - \mathbf{X}(\frac{1}{\|\mathbf{X}\|_2^2}\frac{\partial}{\partial X}(\sqrt{\mathbf{X}^\top\mathbf{X}}))) \tag{23}$$

$$= \frac{\sqrt{d}}{\|\mathbf{X}\|_2}(I - \mathbf{X}\frac{1}{\|\mathbf{X}\|_2}\frac{1}{2\sqrt{\mathbf{X}^\top\mathbf{X}}}\frac{\partial}{\partial X}(\mathbf{X}^\top\mathbf{X})) \tag{24}$$

$$= \frac{\sqrt{d}}{\|\mathbf{X}\|_2}(I - \mathbf{X}\frac{1}{\|\mathbf{X}\|_2^2}\frac{1}{2}2\mathbf{X}^\top) \tag{25}$$

$$= \frac{\sqrt{d}}{\|\mathbf{X}\|_2}(I - \frac{\mathbf{X}\mathbf{X}^\top}{\|\mathbf{X}\|_2^2}) \tag{26}$$

Which is the identical matrix, just written entirely in terms of $\mathbf{X}$.

12. Deja Vu: Denoising diffusion models for generation (28 pts)

In lecture, we talked about denoising diffusion models to get samples from a continuous distribution. This problem is about the potentially simpler binary case. We will assume that we have an unknown distribution of black-and-white images $P(\mathbf{x})$ together with a very large number of example images $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$. Formally, each image can be viewed as a binary vector of length m, i.e. $\mathbf{x} \in \{-1, +1\}^m$.

The first conceptual step in setting up a diffusion model is to choose the easy-to-sample distribution that we want to have at the end of the forward diffusion. For this, we choose m iid fair coin tosses (here we think of a fair coin as having a 50% chance of being +1 and a 50% chance of being -1) arranged into a vector.

Next, we need to choose a way to incrementally degrade the images. Let $\mathbf{Y}_0$ start with whatever image sample $\mathbf{x}$ we want to start with. At diffusion stage t, we generate $\mathbf{Y}_t$ from $\mathbf{Y}_{t-1}$ by randomly flipping each pixel of $\mathbf{Y}_{t-1}$ independently with probability δ where δ is a small positive number.

It turns out that this process of rare pixel-flipping can be reinterpreted for easier analysis. For the j-th pixel at diffusion stage t, this process can alternatively be viewed as first flipping an independent coin $R_t[j]$ with a probability 2δ of coming up $+1$ and then, if $R_t[j] = +1$ replacing $Y_{t-1}[j]$ with a freshly drawn independent fair coin $F_t[j]$ that is equally likely to be -1 or $+1$. If $R_t[j] \neq +1$, we leave that pixel alone i.e. $Y_t[j] = Y_{t-1}[j]$.

(a) (4pts) We need to verify that if we do this and diffuse for sufficiently many stages T, that the resulting distribution is close to looking like m i.i.d. fair coins. **Show that the probability that pixel j has been replaced at some point by an independent fair coin by time T goes to 1 as $T \to \infty$.**

(HINT: It might be helpful to look at the probability that the desired event has not happened…)

Solution: After T stages, consider the probability that the j-th pixel has not been replaced. This requires the T i.i.d. coin tosses to all have come up tails (or -1 if you prefer) and by independence, the probability of that happening is $(1-2\delta)^T$. Since $\delta > 0$, we know that $|1-2\delta| < 1$ and by stability, this implies that $\lim_{T\to\infty}(1-2\delta)^T = 0$. Since the probability of the complementary event is going to zero, we know that the probability that the pixel *has* been replaced by time T is going to 1.

 All Rights Reserved. This may not be publicly shared without explicit permission.

(b) (6pts) To efficiently do diffusion training, we need a way to be able to quickly sample a realization of $\mathbf{Y}_t$ starting from $\mathbf{Y}_0 = \mathbf{x}_i$. **Give a procedure to sample a realization of $\mathbf{Y}_t$ given $\mathbf{Y}_0$ without having to generate $\mathbf{Y}_1, \mathbf{Y}_2, \ldots, \mathbf{Y}_{t-1}$.**

(HINT: Your answer should involve flipping at most two (potentially biased) coins for each pixel.)

Solution: We know from the previous part that that the probability that j-th pixel has not been replaced by a fair coin by time t is $(1-2\delta)^t$. Consequently, we flip a coin with a probability $(1-2\delta)^t$ of coming up tails. (Note: this can be done by sampling a uniform random variable on $[0,1]$ and seeing if it is greater than $(1-2\delta)^t$ and calling that a head.) If it comes up heads, replace the j-th pixel of $\mathbf{Y}_0$ with a freshly tossed independent fair coin (-1 or +1 equally likely) for $y_t[j]$. Otherwise, have $y_t[j] = y_0[j]$.

(c) (2pts) For the reverse diffusion process that will be used during image generation and is being learned during training, our goal is to approximate $P(\mathbf{Y}_{t-1}|\mathbf{Y}_t)$ with a neural net that has learnable parameters θ. Suppose I give you a neural net whose input is a binary image $\mathbf{Y}_t$ and whose output is m real numbers that could each in principle be from $-\infty$ to $+\infty$ (for example, these could be the outputs of a linear layer). **Which of the following nonlinear activation functions would be most appropriate to convert a real number into a probability that we could use to sample whether the pixel in question should be a $+1$?**

◯ Sigmoid $\frac{1}{1+\exp(-x)}$

Solution: The sigmoid is the right answer since since it always returns a number between 0 to 1 which is required for a valid probability.

◯ ReLU $\max(0, x)$

Solution: This is the wrong answer since it can be greater than 1.

◯ Tanh $\tanh(x) = \frac{\exp(2x)-1}{\exp(2x)+1}$

Solution: This is the wrong answer since it can be negative.

(d) (10pts) The goal of training is to approximate a probability distribution for random denoising. However, we do not actually have access to $P(\mathbf{Y}_{t-1}|\mathbf{Y}_t)$ and decide to use $P(\mathbf{Y}_{t-1}|\mathbf{Y}_t, \mathbf{Y}_0)$ instead as a proxy. Note: this can be interpreted as finding an appropriate target distribution that corresponds to trying to predict $\mathbf{Y}_0$ itself at this stage.

What is $P(Y_{t-1}[j] = +1|\mathbf{Y}_t = \mathbf{y}, \mathbf{Y}_0 = \mathbf{x})$?

For simplicity, just do this calculation for the case $x[j] = +1$. To further help you save some time, you may use the following helper result that comes from Bayes' Rule. If A and B are both binary random variables where the prior probabilities are $P(A = +1) = \rho$ and $P(A = -1) = 1 - \rho$, with B being bit-flipped from A with independent probability δ — that is, $P(B = +1|A = +1) = 1 - \delta$, $P(B = -1|A = +1) = \delta$, $P(B = +1|A = -1) = \delta$, and $P(B = -1|A = -1) = 1 - \delta$ — then the conditional probabilities for A conditioned on B are given by:

$$P(A = +1|B = +1) = \frac{(1-\delta)\rho}{(1-\rho)\delta + (1-\delta)\rho} \tag{27}$$

$$P(A = -1|B = +1) = \frac{(1-\rho)\delta}{(1-\rho)\delta + (1-\delta)\rho} \tag{28}$$

$$P(A = +1|B = -1) = \frac{\delta\rho}{\rho\delta + (1-\delta)(1-\rho)} \tag{29}$$

$$P(A = -1|B = -1) = \frac{(1-\delta)(1-\rho)}{\rho\delta + (1-\delta)(1-\rho)} \tag{30}$$

(HINT: What is the distribution for $Y_{t-1}[j]$ given $Y_0[j]$?)

 All Rights Reserved. This may not be publicly shared without explicit permission.

Solution: Following the strategy urged by the hint, we let A be $Y_{t-1}[j]$ and B be $Y_t[j]$ so we can use the formulas above. Since we are interested in the conditional probability of $Y_{t-1}[j] = +1$, the relevant formulas we want to invoke are (27) and (29).

Conditioning on $\mathbf{Y}_0 = \mathbf{x}$ just tells us that $x[j] = 1$ and by the underlying Markovianity of the evolution from $\mathbf{Y}_{t-1} \to \mathbf{Y}_t$, we know that $P(Y_t[j]|\mathbf{Y}_{t-1}, \mathbf{Y}_0) = P(Y_t[j]|\mathbf{Y}_{t-1}) = P(Y_t[j]|Y_{t-1}[j])$. This means that we are exactly in the setup for the calculation given above. This means that all we need is the $\rho = P(Y_{t-1}[j] = +1|Y_0[j] = x[j] = +1)$.

Since ρ is the probability of a 1, there are two disjoint ways we could have a 1. The original 1 could have survived or the final fair coin toss that replaced it was itself a 1.

$$\rho = (1-2\delta)^{t-1} + (1-(1-2\delta)^{t-1})\frac{1}{2} \tag{31}$$

$$= \frac{1+(1-2\delta)^{t-1}}{2} \tag{32}$$

and plugging this into (27) and (29) gives us:

$$P(Y_{t-1}[j] = +1|\mathbf{Y}_t = \mathbf{y}, \mathbf{Y}_0 = \mathbf{x}) = \begin{cases} \frac{(1-\delta)\rho}{(1-\rho)\delta+(1-\delta)\rho} & \text{if } y[j] = +1 \\ \frac{\delta\rho}{\rho\delta+(1-\delta)(1-\rho)} & \text{if } y[j] = -1 \end{cases}.$$

Note that if we had $x[j] = -1$, what would happen is that ρ would be $\frac{1-(1-2\delta)^{t-1}}{2}$.

(e) (6pts) Let the (conditional) probability distribution (on whether each pixel is $+1$) output by our neural net with nonlinearity be $Q_t(\mathbf{Y}_t)$. For training the denoising diffusion model $q(\mathbf{Y}_{t-1}|\mathbf{Y}_t)$, we choose to use SGD loss $D_{KL}(P(\mathbf{Y}_{t-1}|\mathbf{Y}_t, \mathbf{Y}_0 = \mathbf{x}_i)||Q_t(\mathbf{Y}_t))$ where $\mathbf{x}_i$ is the random training image drawn, t is the random time drawn from 1 to T, and $\mathbf{Y}_t$ is the randomly sampled realization of the forward diffusion at time t starting with the image $\mathbf{x}_i$ at time 0. This ends up being a loss on the vector of probabilities coming out of $Q_t(\mathbf{Y}_t)$ that can be written as a sum over the m entries in the vector of probabilities.

Given what you know about KL Divergence, what does this loss penalize most strongly? What does this loss look like at $t = 1$ in particular?

Because there are no cheat-sheets allowed and the order matters, we give you the formula for KL Divergence in the discrete case here: $D_{KL}(P(z)||Q(z)) = \sum_z P(z) \ln \frac{P(z)}{Q(z)}$.

Solution: The KL divergence penalizes most strongly the probability that the neural net says that something is very unlikely when the proxy target distribution has significant probability. This is due to the Q (from the neural net) being the second term in the KL divergence.

It is informative to look at $t = 1$ as a starting point. For $t = 1$, the target distribution we are using is $P(\mathbf{Y}_{1-1}|\mathbf{Y}_1, \mathbf{Y}_0 = \mathbf{x}_i) = P(\mathbf{Y}_0|\mathbf{Y}_1, \mathbf{Y}_0 = \mathbf{x}_i) = P(\mathbf{Y}_0|\mathbf{Y}_0 = \mathbf{x}_i)$ which is always going to be putting all of its probability on $\mathbf{x}_i$. Using the formula for KL divergence, we see that there is only a single term in the expectation and so:

$$D_{KL}(P(\mathbf{Y}_0|\mathbf{Y}_0 = \mathbf{x}_i)||Q_1(\mathbf{Y}_1)) = \log \frac{1}{Q_1(\mathbf{x}_i|\mathbf{Y}_1)} \tag{33}$$

$$= \sum_{j=1}^{m} -\log Q_1(\mathbf{x}_i[j]|\mathbf{Y}_1) \tag{34}$$

which is identical to binary cross-entropy loss for denoising the original binary image, treating every pixel as a binary classification problem.

 All Rights Reserved. This may not be publicly shared without explicit permission.

This insight allows us to, in the binary case, reinterpret the losses at every stage of the denoising diffusion model. After all, in the previous problem we can see that the target distribution for the KL loss is approaching a fair coin toss as $t \to \infty$. This can be interpreted as binary cross-entropy loss for reconstructing the original binary image, except using a label-smoothing schedule where the target labels are increasingly smoothed as t grows.

Under this interpretation, every stage of the reverse diffusion is very much an image denoiser. However, as we increase the level of the noise, we increasingly smooth the labels for the cross-entropy loss function.

This interpretation should help you understand more deeply why substantial weight-sharing across different t makes so much sense — they are all reflecting the same basic pattern which is being learned.

Contributors:

- Lance Mathias.
- Anant Sahai.
- Kevin Frans.
- Gireeja Ranade.
- Sultan Daniels.
- Patrick Mendoza.
- Joey Hong.

 All Rights Reserved. This may not be publicly shared without explicit permission.